

Progetto Blockchain

Tema 4 - Solidity avanzato

Gabriele Lippolis - Fabio Ottico

Introduzione

La blockchain di Ethereum ha subito un significativo sviluppo tecnico negli ultimi anni, risolvendo i limiti storici dell'EVM in termini di scalabilità, costi ed esperienza utente. Lo sviluppo degli smart contract su Ethereum, in passato, presentava alcuni vincoli intrinseci: una limitazione imposta dagli elevati costi di mantenimento dello stato della blockchain e una rigida distinzione tra account con controllo della chiave privata (EOA) e smart contract (SCA). Questo progetto mira a studiare e implementare le soluzioni più recenti progettate per superare tali carenze, concentrandosi sull'intersezione di tre concetti avanzati per lo sviluppo di Solidity e dell'EVM:

- **Transient Storage**: un nuovo concetto di archiviazione introdotto nell'hard fork Dencun. Introduce una regione di memoria transitoria che viene conservata solo per la durata di una transazione, consentendo così un notevole risparmio sulle commissioni di gas ed evitando al contempo aggiornamenti permanenti del Merkle Trie.
- **Inline Assembly (Yul)**: il linguaggio intermedio di basso livello che consente agli sviluppatori di superare l'astrazione dell'EVM, manipolare direttamente la memoria e ottimizzare il codice per operazioni complesse (come i contratti proxy o le chiamate generiche) che non sono facilmente realizzabili con Solidity standard.
- **EIP-7702 e Account Abstraction**: un elemento chiave del futuro modello di transazione di Ethereum, previsto per l'hard fork Pectra. Consente a un EOA esistente di impersonare temporaneamente uno smart contract tramite una firma delegata (una "Set Code Transaction", tipo 0x04), rendendo possibile un comportamento flessibile e programmabile dell'account.

La validità, l'efficienza in termini di gas e la sicurezza dell'architettura proposta sono state valutate attraverso test con il framework Foundry.

Transient Storage

Il transient Storage è un terzo spazio in cui vengono memorizzati i dati nell'EVM, a metà strada tra la memoria volatile e la memoria permanente. Il transient Storage è stato implementato tramite l'EIP-1153 ed è stato aggiunto alla mainnet con l'hard fork Dencun.

In precedenza, i contratti Solidity potevano fare affidamento su:

- **Memory**: una memoria lineare a basso costo, disponibile solo per la durata di un contesto di chiamata. Ogni chiamata esterna (CALL, DELEGATECALL, ecc.) riceve un blocco di memoria separato e isolato, non accessibile al chiamante.
- **Storage**: una mappa chiave-valore persistente (in cui chiavi e valori hanno una larghezza di 32 byte) che può essere modificata tra una transazione e l'altra. La modifica dello storage richiede operazioni di scrittura nel Merkle Trie dello stato globale, con un costo estremamente elevato che può arrivare fino a 20.000 gas per una scrittura a freddo di uno slot.

Con l'EIP-1153 sono stati introdotti gli opcode TSTORE (Transient Store) e TLOAD (Transient Load). Il transient Storage si comporta strutturalmente allo stesso modo dello storage: è una mappa chiave-valore composta da slot di 32 byte di larghezza. Tuttavia, la sua durata è diversa, i dati persistono solo per la durata di una transazione, dopodiché vengono cancellati. Ciò significa che sono

accessibili a tutti i frame di esecuzione nella stessa transazione (comprese le chiamate annidate o le transazioni che coinvolgono più contratti), ma non vengono mai salvati su disco in modo permanente nello storage dello stato globale del nodo validatore.

Poiché i dati non devono essere archiviati in modo permanente al termine della transazione, richiedono risorse minime da parte del nodo. Di conseguenza, il costo in gas per il transient storage è estremamente basso e fisso: 100 gas sia per TSTORE che per TLOAD. A differenza dello storage permanente, dove il costo di una scrittura varia da 2.900 a 20.000 gas a seconda che lo slot sia stato scritto o meno in quella transazione, il transient storage ha una tariffa fissa prevedibile ed economica, senza necessità di rimborsi o di gestione del gas per i rimborsi.

Casi d'uso

1. **Guardie di rientranza ottimizzate:** l'approccio tipico per prevenire attacchi di rientranza nei contratti prevede la scrittura di un valore booleano (o intero) nello storage persistente prima di una chiamata e la sua successiva cancellazione immediatamente dopo:

```
1 // metodo tradizionale in storage permanente (costoso)
2 modifier nonreentrant() {
3     require(_status != _entered, "reentrancyguard: reentrant call");
4     _status = _entered; // sstore
5     _;
6     _status = _not_entered; // sstore
7 }
```

Utilizzando il transient storage, una protezione contro la rientranza può impostare e verificare il flag utilizzando rispettivamente TSTORE e TLOAD, con costi di gas significativamente ridotti e senza necessità di pulizia dopo la transazione. Inoltre, si evitano inutili cambiamenti di stato nello storage e nel Merkle Trie.

2. **Callback senza stato e prestiti flash:** le applicazioni DeFi si basano spesso su prestiti flash e altre operazioni complesse che modificano lo stato, le quali richiedono il passaggio dello stato o la garanzia dell'integrità dell'esecuzione del contratto al contratto chiamante. Prima dell'introduzione del Transient Storage, queste interazioni complesse richiedevano o la memorizzazione dello stato temporaneo nello Storage persistente (con i relativi costi di gas e rischi) o un intricato passaggio di parametri. Il Transient Storage fornisce un meccanismo naturale ed efficiente per i dati di stato temporanei – quali gli importi dei prestiti, i saldi contrattuali previsti o l'identità del richiedente del prestito – che persistono solo per la durata della transazione.
3. **Approvazioni ERC-20 a transazione singola:** per mitigare i rischi associati alle approvazioni illimitate in ERC-20, dove gli utenti potrebbero essere sfruttati attraverso vulnerabilità del contratto approvato, è possibile concedere un'approvazione ERC-20 temporanea che esiste solo per la transazione in cui viene richiesta. Questo approccio, implementato tramite il Transient Storage, elimina la possibilità che un aggressore sottragga fondi in una transazione successiva sfruttando un'approvazione di lunga durata.
4. **Condivisione del contesto in multicall complesse:** le transazioni complesse in batch possono trarre vantaggio da un modo per condividere dati di configurazione temporanei tra varie chiamate. Il Transient Storage offre un mezzo efficiente e sicuro affinché diverse chiamate di contratto all'interno della stessa transazione possano condividere il contesto, come il destinatario finale dei fondi in un batch di swap o i limiti di slippage aggregati, senza ricorrere a costosi argomenti in calldata o a modifiche dello Storage persistente.

Inline Assembly in Solidity

L'inline assembly in Solidity consente agli sviluppatori di scrivere codice a basso livello direttamente nel contratto utilizzando Yul. Yul è un linguaggio intermedio progettato per rappresentare operazioni a basso livello. È costituito da costrutti semplificati quali variabili locali, funzioni, cicli e istruzioni di base che vengono tradotti quasi direttamente in opcode EVM.

L'inline assembly di Solidity utilizza la sintassi Yul.

Tutte le variabili all'interno del blocco assembly rientrano esclusivamente nell'ambito di Yul e non sono soggette al sistema di tipi statico di Solidity.

Mentre i costrutti di alto livello di Solidity forniscono funzionalità come tipi complessi, controllo statico dei tipi, cicli, gestione automatica dell'overflow e dell'underflow aritmetico, l'EVM utilizza uno stack a 256 bit, una Memory EVM con indirizzamento a byte, lo Storage EVM organizzato in slot persistenti da 256 bit e numerosi opcode. Un blocco assembly `{}` consente agli sviluppatori di aggirare alcune protezioni offerte da Solidity, manipolare direttamente lo stack, leggere e scrivere dati nella Memory EVM, accedere allo Storage tramite gli opcode dedicati oppure utilizzare direttamente opcode non esposti dal linguaggio di alto livello.

Vantaggi e Svantaggi

- **Vantaggi:** controllo completo del layout della Memory EVM, consumo ottimizzato di gas, implementazione di funzionalità che non possono essere facilmente scritte in Solidity o che in Solidity risultano proibitive in termini di costo.
- **Svantaggi:** i controlli di sicurezza integrati in Solidity (come i controlli sui limiti degli array e la gestione automatica degli accessi alla memoria) sono disattivati all'interno dell'assembly. Un singolo errore nella gestione degli offset di memoria può portare al danneggiamento dei dati in memoria, a comportamenti imprevisti, al fallimento delle transazioni e a gravi vulnerabilità di sicurezza.

Casi d'uso

1. Allocazione e manipolazione ultraveloci della memoria e gestione manuale del free memory pointer:

Il compilatore utilizza normalmente lo slot situato a 0x40 (noto come free memory pointer) per tenere traccia dell'area di memoria libera disponibile. Per ragioni di efficienza e semplicità di implementazione, il compilatore applica un modello di allocazione standardizzato. Il programmatore in assembly può invece scegliere di allocare manualmente regioni di memoria per le proprie variabili oppure utilizzare direttamente l'area di memoria temporanea riservata allo scratch space (offset da 0x00 a 0x3f) per operazioni intermedie, come hashing e calcoli temporanei, purché tali aree non contengano dati ancora utilizzati dal codice.

Questa tecnica può risultare complessa poiché è necessario evitare di sovrascrivere dati presenti in memoria utilizzati da altre parti del programma.

```
1 assembly {
2   // Recupera il puntatore alla memoria libera
3   let ptr := mload(0x40)
4   // Scrive un valore a quel puntatore
5   mstore(ptr, 0x01)
6   // Aggiorna il free memory pointer manualmente
7   mstore(0x40, add(ptr, 32))
8 }
```

2. Creazione di contratti proxy generici (EIP-1167 / ERC-1967):

Un contratto proxy inoltra tutte le chiamate in ingresso a un contratto di implementazione utilizzando `delegatecall`. Il compilatore Solidity espone `delegatecall`, ma l'implementazione di un contratto proxy completamente trasparente richiede comunque, in genere, l'uso di inline assembly per copiare dinamicamente `calldata` e `returndata`:

```
1 fallback() external payable {
2     assembly {
3         // Copia i calldata in memoria
4         calldatacopy(0, 0, calldatasize())
5         // Esegue la chiamata delegata all'implementazione
6         let result := delegatecall(
7             gas(), sload(implementation), 0, calldatasize(), 0, 0
8         )
9         // Copia i dati restituiti (returndata)
10        returndatacopy(0, 0, returndatasize())
11        // Ritorna il risultato o esegue il revert a seconda dell'esito
12        switch result
13        case 0 { revert(0, returndatasize()) }
14        default { return(0, returndatasize()) }
15    }
16 }
```

3. Distribuzione deterministica tramite CREATE2:

L'opcode `CREATE2` consente di effettuare il deployment di un contratto a un indirizzo prevedibile prima che il relativo creation bytecode venga pubblicato sulla blockchain, sulla base del codice di creazione, di un salt arbitrario e dell'indirizzo del deployer. Il compilatore Solidity supporta ora questa funzionalità tramite la creazione di una nuova istanza del contratto specificando il salt desiderato, ma l'inline assembly rimane una scelta valida per implementare factory generiche ed efficienti in grado di distribuire qualsiasi bytecode precalcolato.

4. Hash e lettura della memoria ad alta velocità:

A volte, il compilatore deve creare più copie temporanee di aree di memoria per preparare i dati necessari affinché `KECCAK256` possa essere eseguito in modo efficiente. Con il linguaggio assembly è possibile calcolare l'hash di qualsiasi blocco contiguo della Memory EVM semplicemente specificando l'offset iniziale e la lunghezza, risparmiando così quantità significative di gas in applicazioni come ponti cross-chain e Merkle Patricia Trie, ad esempio.

EIP-7702, Transazione Type 0x04 e astrazione degli account

Descrizione tecnica dell'EIP-7702

L'EIP-7702, proposto da Vitalik Buterin e implementato nell'hard fork Pectra, rappresenta uno degli aggiornamenti più significativi al modello degli account di Ethereum dalla sua nascita.

Tradizionalmente, la rete divideva rigorosamente gli account in due tipi distinti:

EOA (Externally Owned Account): account standard controllati da una chiave privata (ad esempio, i wallet convenzionali), che non possiedono codice eseguibile. SCA (Smart Contract Account): account governati da codice memorizzato sulla blockchain, privi di una chiave privata associata.

L'EIP-7702 supera questa dicotomia consentendo a un EOA di associare al proprio account un designatore di delega verso un contratto di implementazione, che può essere successivamente aggiornato tramite una nuova autorizzazione.

L'EIP-7702 opera attraverso il modello di delega dell'EVM. L'utente firma un'autorizzazione indicando che il proprio account delega l'esecuzione a uno specifico indirizzo di implementazione. Durante l'elaborazione della transazione, l'EVM inserisce nell'account dell'utente un delegation designation nel formato: `0xef01 || address`

Quando un altro contratto o una transazione interagisce con questo EOA, l'EVM rileva il delegation designation e utilizza il codice del contratto delegato per eseguire le operazioni, mantenendo però lo Storage, il saldo e l'indirizzo dell'EOA delegante. L'EOA assume quindi temporaneamente il comportamento di uno smart account, pur mantenendo la propria identità di EOA, conservando la chiave privata originale per future autorizzazioni ed evitando la necessità di trasferire fondi a un nuovo indirizzo.

La transazione Type 0x04 (Set Code Transaction)

Per facilitare questo meccanismo di delega, l'EIP-7702 introduce un nuovo tipo di transazione standardizzato, identificato dal tipo 0x04, denominato Set Code Transaction.

Il payload codificato RLP di una transazione di tipo 0x04 è strutturato come segue:

```
rlp([
  chain_id,
  nonce,
  max_priority_fee_per_gas,
  max_fee_per_gas,
  gas_limit,
  to,
  value,
  data,
  access_list,
  authorization_list,
  signature_y_parity,
  signature_r,
  signature_s,
])
```

L'innovazione chiave risiede nell'`authorization_list`, che consiste in un array di autorizzazioni firmate:

```
authorization_list =
[
  [
    chain_id,
    address,
    nonce,
    y_parity,
    r,
    s,
  ],
  ...
]
```

- **address**: l'indirizzo del contratto delegato che l'EOA desidera utilizzare come logica di esecuzione.
- **signature (y_parity, r, s)**: la firma crittografica generata dalla chiave privata dell'EOA delegante.

Ogni autorizzazione è indipendente, consentendo all'EVM di elaborare più autorizzazioni all'interno di una singola transazione. Flusso di esecuzione:

Quando viene inviata una transazione Type 0x04, la rete convalida la firma dell'EOA per ciascuna autorizzazione presente nell'`authorization_list`. Se la firma è valida, aggiorna il `delegation designation` dell'account impostandolo al valore `0xef01 || address` per quell'EOA. In particolare, una transazione Type 0x04 può essere inviata e pagata da chiunque, purché contenga una firma valida presente nell'elenco delle autorizzazioni; non è necessario che il mittente della transazione sia il titolare dell'EOA stesso.

Casi d'uso dell'Account Abstraction basati su EIP-7702

L'EIP-7702 funge da catalizzatore per l'Account Abstraction (AA), integrandosi in modo nativo con l'infrastruttura basata su ERC-4337 (bundler, paymaster). Tra i principali casi d'uso figurano:

1. **Batching di transazioni atomiche:**

In un'applicazione DeFi tradizionale, l'interazione con un protocollo (ad esempio, lo swap di token ERC-20 su Uniswap) richiede generalmente agli utenti di firmare e pagare il gas per due transazioni separate: l'approvazione e successivamente lo swap stesso. Con l'EIP-7702, l'EOA può delegare temporaneamente l'esecuzione a un contratto multicall. L'utente firma un'unica

transazione di tipo 0x04, che consente di eseguire in modo atomico e sicuro sia l'approvazione sia il trasferimento all'interno della stessa transazione, riducendo i costi di gas complessivi e migliorando significativamente l'esperienza utente.

2. Sponsorizzazione del gas e paymaster:

Molti utenti hanno difficoltà ad accedere al Web3 perché non dispongono di ETH nativo per sostenere i costi di gas, anche se possiedono stablecoin (come USDC) o NFT. Grazie alla transazione di tipo 0x04, un'applicazione di terze parti o un paymaster può sottomettere la transazione di un utente alla blockchain, anticipando i costi di gas in ETH. Il contratto delegato utilizzato temporaneamente dall'EOA dell'utente può quindi dedurre l'equivalente del costo di gas dal saldo USDC dell'utente per rimborsare il paymaster, consentendo un'esperienza utente gasless o con pagamento flessibile delle commissioni.

3. Chiavi di sessione e riduzione dei privilegi:

Nel contesto dei giochi su blockchain o del trading ad alta frequenza, richiedere una firma manuale per ogni azione compromette l'esperienza utente o l'efficienza operativa. Con l'EIP-7702, un EOA può delegare temporaneamente l'esecuzione a un contratto che implementa funzionalità da Smart Account e autorizzare una session key con privilegi limitati. Ad esempio, un utente potrebbe creare un'autorizzazione specificando:

“La chiave di sessione X è autorizzata a eseguire operazioni per mio conto, ma solo per interagire con lo smart contract di gioco Y, spendendo al massimo 10 token al giorno, per un periodo massimo di 2 ore”.

Se la chiave di sessione viene compromessa, i fondi principali dell'utente rimangono al sicuro.

1. Recupero sociale retroattivo per gli EOA:

Il rischio più significativo per gli utenti Web3 è la perdita della propria seed phrase associata all'EOA. Fino ad ora, gli utenti dovevano migrare tutte le risorse verso nuovi indirizzi basati su Smart Account per beneficiare di sistemi di recupero multisig o basati su guardian. Con l'EIP-7702, gli utenti possono aggiornare retroattivamente i propri EOA esistenti associandoli a una logica di Smart Account tramite un contratto delegato che implementa funzionalità di recupero sociale. Questa delega può essere modificata successivamente tramite una nuova autorizzazione.

La logica dello Smart Account può implementare un meccanismo di recupero basato su guardian, consentendo la rotazione della chiave di controllo e collegando l'EOA dell'utente a una nuova chiave pubblica senza modificare l'indirizzo originale o le risorse associate.

Strumenti di sviluppo – Il framework Foundry

Che cos'è Foundry e perché è stato scelto

Per lo sviluppo, il collaudo e l'analisi delle prestazioni degli smart contract implementati nel progetto, è stato adottato Foundry, un toolkit di sviluppo per applicazioni Ethereum scritto interamente in Rust.

A differenza dei framework tradizionali come Hardhat o Truffle, che delegano gran parte dell'automazione a linguaggi esterni come JavaScript o TypeScript, Foundry consente di scrivere test, script di distribuzione e procedure di automazione direttamente in Solidity.

Questa caratteristica offre diversi vantaggi:

1. Riduzione del cambio di contesto: lo sviluppatore opera interamente all'interno dell'ecosistema Solidity, riducendo la complessità cognitiva e il rischio di errori dovuti al continuo passaggio da un linguaggio all'altro.
2. Prestazioni elevate: sviluppato in Rust, Foundry offre tempi di compilazione ed esecuzione significativamente più rapidi rispetto alle principali alternative, consentendo l'esecuzione di suite di test estese in pochi secondi.
3. Conformità all'ambiente di produzione: il framework utilizza direttamente il compilatore solc, garantendo che il bytecode generato durante lo sviluppo corrisponda a quello effettivamente distribuito sulla blockchain.

Forge è il componente centrale del framework ed è responsabile della compilazione dei contratti, dell'esecuzione dei test, dell'analisi della copertura del codice e della profilazione del consumo di gas.

Funzionalità avanzate

Cheatcode

Attraverso la libreria `forge-std`, Foundry mette a disposizione una serie di primitive speciali accessibili tramite l'istanza `vm`, che consentono la manipolazione diretta dello stato dell'EVM durante i test.

Tra le funzionalità più comunemente utilizzate:

- `vm.warp()` per modificare il timestamp del blocco;
- `vm.roll()` per alterare il numero del blocco;
- `vm.prank()` per impersonare un mittente arbitrario;
- `vm.sign()` per generare firme crittografiche;
- `vm.expectRevert()` per verificare le condizioni di errore.

Gestione delle versioni dell'EVM

Il framework consente di specificare con precisione l'hard fork di riferimento tramite il file di configurazione `foundry.toml`, garantendo la compatibilità con gli opcode, i precompilati e gli standard introdotti nelle versioni più recenti della Ethereum Virtual Machine.

Analisi del gas

Per avere i valori del gas speso abbiamo usato il comando `forge test --gas-report` che restituisce il consumo minimo, massimo e medio di gas di ciascuna funzione.

Progettazione di alto livello del progetto

Questo progetto definisce uno Smart Account conforme allo standard EIP-7702 (DeFiSmartAccount). Questo contratto funge da contratto di implementazione che qualsiasi EOA standard può utilizzare dinamicamente tramite una delega impostata con una transazione Type 0x04 (Set Code Transaction).

Per massimizzare la sicurezza e l'efficienza nell'esecuzione di operazioni finanziarie multiple (DeFi batching), utilizziamo:

1. **Un Transient Reentrancy Guard**: protegge dagli exploit di rientranza con un costo di gas inferiore rispetto a una protezione basata sullo Storage persistente.
2. **Una libreria di inline assembly a basso livello (AssemblyUtils)**: facilita la manipolazione della memoria EVM e l'esecuzione sicura di chiamate arbitrarie.
3. **Un Session Verification Guard**: tutela l'EOA dall'esecuzione indesiderata di callback al di fuori di una sessione attiva autorizzata.

Analisi dettagliata dei singoli file

src/1_TransientStorage.sol - TransientReentrancyGuard

Questo contratto astratto implementa un meccanismo di esclusione reciproca (lock) per prevenire attacchi di rientranza utilizzando gli opcode introdotti dall'EIP-1153.

```
1 abstract contract TransientReentrancyGuard {
2     bytes32 private constant REENTRANCY_SLOT = keccak256("reentrancy.guard");
3     error ReentrancyDetected();
4 }
```

1. **REENTRANCY_SLOT**: la chiave dello slot del Transient Storage in cui viene salvato lo stato del blocco. Viene calcolata in modo deterministico tramite keccak256 applicato a una stringa identificativa univoca, riducendo il rischio di collisioni con altri slot utilizzati dal contratto.

Modificatore nonReentrant (Analisi Yul):

```
1     modifier nonReentrant() {
2         bytes32 slot = REENTRANCY_SLOT;
3         assembly {
4             if tload(slot) {
5                 mstore(0x00, 0x5a1532f3)
6                 revert(0x1c, 0x04)
7             }
8             tstore(slot, 1)
9         }
10        _;
11        assembly {
12            tstore(slot, 0)
13        }
14    }
```

- **tload(slot)**: questa istruzione legge il valore memorizzato nello slot del Transient Storage. Se restituisce un valore diverso da zero (cioè 1), significa che una chiamata rientrante ha tentato di rieseguire il codice protetto prima del completamento della chiamata precedente.
- **Gestione del revert a basso livello**: se viene rilevata una rientranza, l'esecuzione deve essere interrotta. Anziché utilizzare le funzionalità ad alto livello di Solidity, si ricorre all'assembly per costruire manualmente il dato di revert:
 1. 0x5a1532f3 rappresenta i primi 4 byte dell'hash di ReentrancyDetected(), ovvero il selettore dell'errore personalizzato.
 2. mstore(0x00, 0x5a1532f3) scrive questo valore nell'area di memoria con offset 0x00. Poiché mstore scrive blocchi da 32 byte (256 bit), il valore viene allineato a destra (preceduto da 28 byte di zeri). I 4 byte significativi si trovano quindi negli ultimi 4 byte del blocco, ovvero tra gli offset 0x1c e 0x20.
 3. revert(0x1c, 0x04) interrompe l'esecuzione restituendo esattamente i 4 byte a partire dall'offset di memoria 0x1c, corrispondenti al selettore dell'errore.
- **tstore(slot, 1)**: se non viene rilevata alcuna rientranza, lo stato del lock viene impostato a 1, impedendo ulteriori ingressi nella sezione protetta durante l'esecuzione.
- **tstore(slot, 0)**: al termine dell'esecuzione, il lock viene nuovamente impostato a 0.
- **_**: rappresenta l'esecuzione del corpo della funzione protetta dal modificatore.

src/2_InlineAssembly.sol - AssemblyUtils

Questa libreria funge da motore computazionale a basso livello del sistema, fornendo funzioni Yul altamente ottimizzate per la gestione della Memory EVM e l'esecuzione di chiamate esterne.

generateSessionHash:

```
1    function generateSessionHash(  
2        address sender, uint256 _timestamp  
3    ) internal pure returns (bytes32 sessionHash) {  
4        assembly {  
5            let ptr := mload(0x40)  
6            mstore(ptr, sender)  
7            mstore(add(ptr, 0x20), _timestamp)  
8            sessionHash := keccak256(ptr, 0x40)  
9        }  
10    }
```

- **mload(0x40)**: legge il valore contenuto nello slot 0x40 della memoria, che in Solidity contiene il free memory pointer. Questo valore rappresenta l'offset da cui è possibile allocare nuova memoria senza sovrascrivere dati già utilizzati.
- **mstore(ptr, sender)**: scrive l'indirizzo del mittente nella posizione di memoria indicata da ptr. Poiché mstore scrive sempre 32 byte, l'indirizzo address (20 byte) viene esteso a 32 byte tramite zero-padding a sinistra.
- **mstore(add(ptr, 0x20), _timestamp)**: calcola l'offset di memoria successivo aggiungendo 32 byte (0x20 in esadecimale) a ptr e memorizza il timestamp in quella posizione.
- **keccak256(ptr, 0x40)**: calcola l'hash Keccak-256 dei 64 byte (0x40 in esadecimale) presenti consecutivamente a partire da ptr, composti dall'indirizzo del mittente e dal valore di _timestamp.

executeLowLevelCall:

```
1    function executeLowLevelCall(  
2        address target, uint256 value, bytes memory data  
3    ) internal returns (bool success) {  
4        assembly {  
5            let dataPtr := add(data, 0x20)  
6            let dataLen := mload(data)  
7            success := call(gas(), target, value, dataPtr, dataLen, 0, 0)  
8        }  
9    }
```

- **Layout di memoria dei dati bytes**: in Solidity, un array dinamico di byte (bytes) in memoria segue un layout specifico: i primi 32 byte contengono la lunghezza dell'array, mentre i dati effettivi del payload iniziano immediatamente dopo, all'offset successivo.
- **dataLen := mload(data)**: legge la lunghezza dei dati caricando i primi 32 byte dell'area di memoria indicata da data.
- **dataPtr := add(data, 0x20)**: incrementa l'offset di memoria di 32 byte (0x20) per saltare il campo della lunghezza e ottenere l'offset iniziale del payload effettivo della chiamata.
- **call(...)**: esegue una chiamata EVM di basso livello passando il gas rimanente (gas()), l'indirizzo del contratto di destinazione (target), il valore da trasferire espresso in wei (value), l'offset dei dati di input (dataPtr) e la loro lunghezza (dataLen). I parametri relativi ai dati restituiti (return data offset e return data size) sono impostati a 0, 0, quindi qualsiasi valore restituito dalla chiamata viene ignorato.

src/3_DeFiSmartAccount.sol - DeFiSmartAccount

Questo è lo smart contract principale. Quando un utente utilizza EIP-7702 per delegare l'esecuzione del proprio EOA a un contratto che implementa funzionalità da Smart Account, questo codice definisce la logica esecutiva dell'account delegato.

```
1 contract DeFiSmartAccount is TransientReentrancyGuard {
2     bytes32 private constant SESSION_SLOT =
        keccak256("academic.session.active");
```

- **SESSION_SLOT**: slot del Transient Storage dedicato alla gestione dello stato della sessione attiva.

executeDeFiBatch:

```
1     function executeDeFiBatch(
2         Call[] calldata calls
3     ) external payable nonReentrant {
4         bytes32 sessionHash = AssemblyUtils.generateSessionHash(
5             msg.sender, block.timestamp
6         );
7         bytes32 slot = SESSION_SLOT;
8         assembly {
9             tstore(slot, sessionHash)
10        }
11
12        for (uint256 i = 0; i < calls.length; i++) {
13            bool success = AssemblyUtils.executeLowLevelCall(
14                calls[i].target, calls[i].value, calls[i].data
15            );
16            if (!success) revert BatchExecutionFailed(i);
17        }
18
19        assembly {
20            tstore(slot, 0)
21        }
22    }
```

- **Avvio della sessione**: genera un hash crittografico della sessione basato sul mittente (msg.sender) e sul timestamp del blocco. Questo valore viene memorizzato nel Transient Storage utilizzando TSTORE.
- **Esecuzione in batch**: esegue un ciclo sulle chiamate incluse nel batch (ad esempio, prima un'approvazione ERC-20 e successivamente una chiamata di swap su un DEX). Utilizza la funzione a basso livello AssemblyUtils.executeLowLevelCall. Se una qualsiasi chiamata fallisce, l'intera transazione viene annullata tramite revert, garantendo l'atomicità dell'operazione.
- **Fine sessione**: reimposta lo slot del Transient Storage a 0. In ogni caso, il Transient Storage viene automaticamente cancellato al termine della transazione.

Meccanismo di sicurezza: _verifySession, receive e fallback

```
1     function _verifySession() internal view {
2         bytes32 slot = SESSION_SLOT;
3         assembly {
4             let activeSession := tload(slot)
5             if iszero(activeSession) {
6                 mstore(0x00, 0x1e360fbc)
```

```

7             revert(0x1c, 0x04)
8         }
9     }
10 }
11
12 receive() external payable { _verifySession(); }
13 fallback() external payable { _verifySession(); }

```

Quando un utente delega il proprio EOA a questo contratto tramite EIP-7702, l'account utilizza il codice delegato per gestire le chiamate ricevute, incluse le chiamate che attivano `receive()` o `fallback()`. Questo introduce un potenziale rischio di sicurezza: un aggressore (o un contratto malevolo con cui l'utente interagisce) potrebbe tentare di provocare un'esecuzione non autorizzata tramite callback inattesi, ad esempio nel contesto di token ERC-777 o di meccanismi reentranti.

La funzione `_verifySession()` protegge questi ingressi leggendo lo slot del Transient Storage tramite `TLOAD`. Se il valore letto è 0 (ovvero non esiste una sessione attiva inizializzata da `executeDeFiBatch`), l'esecuzione viene interrotta tramite un `revert` con l'errore custom `SessionNotActive()` (0x1e360fbc).

src/mocks/MockDEX.sol e MockERC20.sol

Questi contratti simulano un'infrastruttura DeFi reale per validare il comportamento del sistema nei test:

- **MockERC20**: un semplice token ERC-20 che implementa le funzionalità standard di minting, approvazione e trasferimento.
- **MockDEX**: simula un exchange decentralizzato (DEX). Preleva token dall'utente tramite `transferFrom` e trasferisce in cambio 1 ETH utilizzando una chiamata EVM a basso livello:
`(bool success, ) = msg.sender.call{value: 1 ether}("");`

Nota importante: questa chiamata low-level verso `msg.sender` (che nei test corrisponde all'EOA di Alice delegata) attiva la funzione `receive()` del codice delegato associato all'EOA tramite EIP-7702.

Analisi della suite di test (test/DeFiSmartAccount.t.sol)

La suite di test in Foundry verifica il corretto funzionamento del sistema sfruttando il meccanismo di autorizzazione tramite firma introdotto da EIP-7702 attraverso i cheatcode nativi di Foundry.

Integrazione di EIP-7702 in Foundry

I test definiscono una chiave privata di test per Alice (`alicePrivateKey`) e ne ricavano l'indirizzo EOA:

```
1 aliceEOA = vm.addr(alicePrivateKey);
```

Il cheatcode fondamentale utilizzato nei test è **`vm.signAndAttachDelegation`**:

```
1 vm.signAndAttachDelegation(address(implementation), alicePrivateKey);
```

Questo comando di Foundry simula il meccanismo di delega di EIP-7702. Esso genera e applica un'autorizzazione firmata con la chiave privata di Alice, indicando il contratto di implementazione (`DeFiSmartAccount`) come codice delegato dell'EOA. Da quel momento, nell'ambiente di test, quando l'indirizzo di Alice (`aliceEOA`) viene utilizzato come account esecutore, l'EVM utilizza la logica del contratto delegato.

Spiegazione di ciascun test

TestEIP7702DeFiBatching (Batching Multicall)

Questo test verifica se è possibile eseguire più operazioni DeFi in modo atomico all'interno di un'unica transazione autorizzata:

- Alice firma un'autorizzazione EIP-7702 delegando l'esecuzione al contratto di implementazione.
- Prepara un batch composto da due chiamate: prima esegue un'approvazione ERC-20 (MockERC20.approve) verso un DEX, poi effettua lo scambio dei token con ETH (MockDEX.swapTokensForEth).
- Alice esegue la funzione executeDeFiBatch direttamente dal proprio indirizzo EOA (aliceEOA).
- Risultato: il test ha esito positivo. I token vengono trasferiti, Alice riceve 1 ETH e le due operazioni, che normalmente richiederebbero transazioni separate per un EOA tradizionale, vengono eseguite atomicamente all'interno di un'unica transazione.

TestRevertWhenFallbackCalledOutsideSession (Prevenzione del dirottamento)

Questo test dimostra l'efficacia del controllo di sessione basato sul Transient Storage:

- La delega EIP-7702 di Alice viene attivata.
- Un attore malevolo (0xBAD) tenta di inviare ETH direttamente all'EOA di Alice (aliceEOA) senza avviare una sessione DeFi batch.
- Risultato: la transazione fallisce con l'errore SessionNotActive(). Questo dimostra che, sebbene l'EOA di Alice utilizzi ora un codice delegato capace di gestire chiamate ricevute, il percorso di esecuzione tramite receive() viene bloccato quando non è presente una sessione attiva autorizzata.

TestRevertWhenReentrancyAttempted (Protezione contro la rientranza)

Questo test verifica la protezione contro gli attacchi di rientranza:

- Viene utilizzato un MaliciousToken che, durante un tentativo di trasferimento ERC-20, cerca di effettuare una chiamata rientrante a executeDeFiBatch verso l'EOA di Alice (aliceEOA).
- Risultato: il modificatore nonReentrant intercetta la seconda invocazione tramite una verifica sul Transient Storage utilizzando l'opcode TLOAD e impedisce la riesecuzione della sezione protetta. L'esecuzione termina con un errore BatchExecutionFailed.

TestEIP7702BatchWithEthTransfer (Flessibilità dei batch)

Questo test verifica la capacità dello Smart Account di gestire scenari complessi, inclusi trasferimenti di ETH verso contratti di terze parti all'interno dello stesso batch DeFi.

TestRevertWhenBatchPartialFailure (Atomicità dello stato)

Questo test verifica che, se una qualsiasi chiamata all'interno di un batch fallisce (in questo caso un DEX viene configurato intenzionalmente per restituire un errore), l'intera esecuzione del batch venga annullata tramite revert e i saldi dell'utente rimangano invariati. Questo garantisce l'atomicità dell'operazione ed evita perdite parziali di fondi.

TestGasComparisonBatchedVsSeparate (Efficienza del gas)

Questo test confronta empiricamente il consumo di gas tra due modalità di esecuzione:

- Caso A (Tradizionale): Alice esegue l'approvazione ERC-20 e lo swap dei token come due transazioni separate dal proprio EOA standard.
- Caso B (EIP-7702): Alice esegue le stesse operazioni aggregate all'interno di un'unica esecuzione batch tramite lo Smart Account delegato.

- Risultato: il test calcola la differenza di gas e conferma che l'approccio batch utilizza meno gas, poiché elimina il costo intrinseco aggiuntivo di una seconda transazione (21.000 gas per una transazione base) e sfrutta operazioni più efficienti implementate tramite inline assembly/Yul.

TestEIP7702RevokeDelegation (Revocabilità del codice)

Questo test affronta uno dei punti più rilevanti relativi a EIP-7702: la possibilità per un utente di rimuovere o modificare la delega associata al proprio account e tornare al comportamento di un EOA senza codice delegato.

- Alice firma una delega iniziale ed esegue con successo un batch.
- Successivamente, Alice firma una nuova autorizzazione impostando l'indirizzo zero (address(0)) come implementazione delegata.
- Questa operazione aggiorna la delegation designation dell'EOA, rimuovendo il riferimento al precedente contratto di implementazione.
- Risultato: il test dimostra che, se Alice tenta di richiamare nuovamente executeDeFiBatch, non viene più risolta alcuna implementazione delegata. Poiché in Ethereum una chiamata verso un EOA senza codice esegue nessuna logica e termina con successo, l'approvazione del token non viene effettuata. Alice torna quindi a comportarsi come un EOA senza delega attiva, in grado di effettuare normali trasferimenti di Ether.

Riepilogo dei concetti chiave applicati nel codice

Concetto teorico	File di riferimento	Riga / Istruzione chiave	Spiegazione dell'applicazione pratica
transient Storage (EIP-1153)	1_TransientStorage.sol, 3_DeFiSmartAccount.sol	tstore(), tload(), SESSION_SLOT	Utilizzata per creare una protezione contro la rientranza ultra-efficiente in termini di gas (100 gas) e una verifica della sessione che protegge il metodo receive() dell'EOA solo nell'ambito del batch DeFi.
Assemblaggio inline (Yul)	2_InlineAssembly.sol, 1_TransientStorage.sol	mload(0x40), mstore(), call(), revert(0x1c, 0x04)	Utilizzato per la manipolazione diretta della memoria a fini di hashing, per chiamate di basso livello che aggirano le limitazioni di Solidity e per la generazione

Concetto teorico	File di riferimento	Riga / Istruzione chiave	Spiegazione dell'applicazione pratica
			personalizzata di errori con un costo di gas di revert minimo.
EIP-7702 e tipo di transazione 0x04	test/DeFiSmartAccount.t.sol	vm.signAndAttachDelegation	Simula la firma della tupla dell'elenco di autorizzazioni. Consente all'EOA di Alice di eseguire il codice nel DeFiSmartAccount mantenendo il proprio indirizzo e saldo.
Astrazione dell'account (Casi d'uso)	test/DeFiSmartAccount.t.sol	testEIP7702DeFiBatching, testEIP7702RevokeDelegation	Dimostra l'attivazione di multicall atomiche (batching), la revoca della delega per tornare all'EOA di origine e la protezione contro attacchi esterni.

Validazione Sperimentale e Analisi dei Consumi di Gas

In questo capitolo vengono presentati e analizzati i risultati della suite di test sviluppata tramite il framework Foundry. L'obiettivo della fase sperimentale è duplice: da un lato, verificare la correttezza logica e la sicurezza dell'architettura proposta (DeFiSmartAccount basato su EIP-7702 ed EIP-1153); dall'altro, quantificare l'efficienza in termini di gas (espressa in unità di gas EVM) delle soluzioni implementate rispetto ai paradigmi tradizionali.

Validazione Funzionale del Ciclo di Vita e della Sicurezza

La suite di test esegue con successo tutti gli 8 test previsti, dimostrando la stabilità delle transizioni di stato e il rispetto dei vincoli di sicurezza imposti dall'architettura.

```

1 Ran 8 tests for test/DeFiSmartAccount.t.sol:
2 DeFiSmartAccountTest [PASS]
3 testEIP7702BatchWithEthTransfer() (gas: 231211) [PASS]
4 testEIP7702DeFiBatching() (gas: 82200) [PASS]
5 testEIP7702RevokeDelegation() (gas: 208915) [PASS]
6 testGasComparisonBatchedVsSeparate() (gas: 212876) [PASS]
7 testGasReportDirectCall() (gas: 36352) [PASS]
8 testRevertWhenBatchPartialFailure() (gas: 404118) [PASS]
```

```
9 testRevertWhenFallbackCalledOutsideSession() (gas: 20803) [PASS]
10 testRevertWhenReentrancyAttempted() (gas: 1011845)
11 Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 3.58ms
```

Dall'analisi dell'esito dei test emergono le seguenti considerazioni sulla sicurezza:

1. **Protezione dalla rientranza (testRevertWhenReentrancyAttempted):** L'intercettazione del tentativo di rientranza da parte del MaliciousToken conferma la validità del modificatore nonReentrant basato su TSTORE/TLOAD. La transazione viene interrotta non appena viene rilevata una violazione del lock transitorio, ripristinando lo stato dell'intera transazione.
2. **Protezione dei canali di ricezione (testRevertWhenFallbackCalledOutsideSession):** Il test fallisce intenzionalmente (con esito positivo per la suite) quando un attore terzo tenta di inviare ETH all'EOA con delega EIP-7702 attiva al di fuori di un batch autorizzato.

Il consumo di gas registrato per questa transazione fallita è particolarmente ridotto (20.803 gas in totale). Questo indica che la logica di controllo `_verifySession()` interviene nelle prime fasi dell'esecuzione, evitando lo spreco di risorse computazionali associato a un'esecuzione più lunga prima del revert.

3. **Atomicità del batch (testRevertWhenBatchPartialFailure):** In caso di fallimento di una chiamata all'interno del batch (ad esempio, un DEX che non soddisfa i requisiti di esecuzione), l'intera esecuzione viene annullata tramite revert, con rollback completo dello stato del token ERC-20 e dei saldi di Alice. Questo comportamento è essenziale per garantire la consistenza patrimoniale dell'utente.

Analisi Comparativa dei Consumi di Gas La metrica pi rilevante per valutare l'efficacia dell'EIP-7702 combinato con l'EIP-1153 il risparmio di gas.

Nel test `testGasComparisonBatchedVsSeparate`, la suite confronta l'esecuzione sequenziale di due transazioni distinte (approvazione e swap) rispetto all'esecuzione aggregata in un'unica chiamata batch tramite lo Smart Account delegato.

I log rilevati riportano:

- Gas transazioni separate (esecuzione EVM): 65.527 gas
- Gas batch (esecuzione EVM): 64.877 gas
- Risparmio netto misurato dall'EVM: 650 gas

Distinzione accademica tra gas di esecuzione e gas di transazione

È di fondamentale importanza evidenziare una sottigliezza tecnica relativa al modo in cui Foundry misura i consumi tramite `gasleft()`. Il risparmio di 650 gas registrato nei log rappresenta esclusivamente l'ottimizzazione del codice durante l'esecuzione all'interno della macchina virtuale (EVM execution gas).

Questo risparmio interno è attribuibile principalmente a:

- L'utilizzo della libreria a basso livello `AssemblyUtils`, che manipola direttamente la memoria EVM per l'hashing e l'esecuzione delle chiamate, riducendo l'overhead introdotto dal compilatore Solidity.
- L'impiego del Transient Storage per gestire lo stato della sessione e i controlli anti-rientranza, evitando operazioni di lettura e scrittura sullo Storage persistente tramite `SLOAD/SSTORE`.

Tuttavia, nello scenario reale on-chain, il risparmio economico per l'utente è significativamente superiore.

Ogni transazione inviata sulla rete Ethereum richiede un costo intrinseco minimo di 21.000 gas (transaction intrinsic gas), oltre ai costi associati ai dati inviati tramite calldata e all'esecuzione del codice EVM.

Se l'utente opera in modalità tradizionale, deve firmare e inviare due transazioni distinte:

- Gas Totale Tradizionale = 21.000 (Transaction Intrinsic Gas TX 1) + Esecuzione 1 + 21.000 (Transaction Intrinsic Gas TX 2) + Esecuzione 2

Se l'utente utilizza un'esecuzione batch abilitata da EIP-7702:

- Gas Totale EIP-7702 = 21.000 (Transaction Intrinsic Gas) + Costo dell'authorization_list + Esecuzione Batch

Evitando la seconda transazione, l'utente elimina il relativo costo intrinseco di 21.000 gas, sostenendo invece un costo aggiuntivo dovuto all'autorizzazione EIP-7702 contenuta nell'authorization_list.

Il risparmio effettivo sulla rete dipende quindi dal contenuto della delega, dal costo della calldata e dalla complessità dell'esecuzione. In scenari tipici con più operazioni DeFi aggregate, il vantaggio può essere nell'ordine di circa 20.000 gas o superiore rispetto all'esecuzione tramite transazioni separate.

Analisi dei Costi di Distribuzione (Deployment)

Il report di Foundry evidenzia inoltre i costi di compilazione e deployment dei contratti di supporto:

Contratto	Deployment Cost (Gas)	Deployment Size (Byte)
MockERC20	227.058	830
MockDEX	185.251	662

I valori di deployment inferiori ai 1.000 byte e con un costo ampiamente inferiore ai 250.000 gas indicano una struttura estremamente compatta.

Nel caso specifico di un'applicazione basata su EIP-7702, questo dettaglio è cruciale: il contratto DeFiSmartAccount non deve essere distribuito individualmente per ogni utente che desidera adottarlo. L'implementazione viene distribuita una sola volta sulla rete come contratto condiviso (singleton implementation), e gli EOA possono associare una delegation designation verso tale implementazione tramite il formato `0xef01 || address`.

Questo elimina il costo di deployment iniziale dello smart account per ogni utente finale, superando uno dei principali svantaggi economici delle architetture basate su smart contract wallet tradizionali, nelle quali l'utente può dover sostenere costi di creazione del proprio account contratto.

Gestione del Ciclo di Vita: Revoca della Delega

Gestione del Ciclo di Vita: Revoca della Delega

Il test `testEIP7702RevokeDelegation` (208.915 gas) convalida sperimentalmente la caratteristica di reversibilità del protocollo. Una volta completate le operazioni DeFi complesse, l'utente può rimuovere la delega associata al proprio account e ripristinare il comportamento standard di un EOA.

Firmando una nuova autorizzazione EIP-7702 che imposta l'indirizzo nullo (`address(0)`) come implementazione delegata, la delegation designation associata all'account viene rimossa.

I test confermano che, in seguito alla revoca:

- I tentativi di invocare funzioni dello Smart Account (come `executeDeFiBatch`) non producono effetti sul token ERC-20 di Alice. La chiamata termina con successo a livello EVM, ma non viene eseguita alcuna logica applicativa, coerentemente con il comportamento standard degli EOA privi di codice delegato.
- L'account di Alice ritorna immediatamente a comportarsi come un EOA convenzionale, in grado di effettuare normali trasferimenti di ETH verso altri account (come convalidato dal trasferimento a Bob).

Questa flessibilità dimostra che EIP-7702 non vincola permanentemente l'utente a uno smart contract, ma consente una gestione modulare e reversibile dei privilegi di esecuzione associati all'account.

Considerazioni Finali sui Risultati

I dati emersi dalla validazione sperimentale confermano che l'integrazione di Transient Storage, inline assembly (Yul) ed EIP-7702 abilita un'architettura altamente efficiente per l'interazione on-chain.

Il sistema dimostra di offrire protezione contro le vulnerabilità di rientranza e contro l'esecuzione non autorizzata tramite gli handler di ricezione (`receive/fallback`), riducendo al contempo i costi operativi per l'utente finale grazie all'esecuzione atomica dei batch e all'eliminazione del costo di deployment individuale dello smart account.

I risultati sperimentali convalidano pertanto l'ipotesi progettuale, evidenziando la fattibilità tecnologica e la sostenibilità economica del modello proposto.